

INTERNSHIP PROJECT REPORT

EMPLOYEE MANAGEMENT SYSTEM WEB APP ON SPRING

Java Spring Boot + Maven + Thymeleaf + HTML/CSS/JavaScript

Internship Project 2

Academic Year: 2026

CERTIFICATE

This is to certify that the internship project report entitled “Employee Management System Web App on Spring” has been prepared as part of internship project work. The project demonstrates the development of a web application using Java Spring Boot, Maven, Thymeleaf, HTML, CSS, and JavaScript.

The application provides employee management functionality through CRUD operations. It supports creating, reading, updating, and deleting employee records and also demonstrates REST API endpoints for employee data. The report presents the project requirements, design, methodology, source code, testing, and sample results.

ACKNOWLEDGEMENT

I sincerely thank my internship mentors, teachers, and everyone who supported me while developing the Employee Management System. Their guidance helped me understand Spring Boot application architecture, Maven dependency management, database persistence, MVC patterns, Thymeleaf templates, and frontend interaction.

This project provided practical exposure to the development of a complete web application. It helped me connect Java programming with browser-based interfaces and understand how CRUD operations move through controllers, services, repositories, JPA, and the database.

I am also thankful for the Java and Spring ecosystem and its documentation, which made it possible to explore reusable components and standard approaches to web application development.

TABLE OF CONTENTS

No.	Section
1	Abstract
2	Objective
3	Introduction
4	Problem Statement and Scope
5	Technology and Software Requirements
6	Technology Overview
7	System Requirements
8	Methodology
9	System Architecture
10	Database Design
11	CRUD API Design
12	Frontend and UI Design
13	Project Structure
14	Implementation Details
15	Code – Entity and Repository
16	Code – Service and MVC Controller
17	Code – REST CRUD Controller
18	Code – Thymeleaf HTML Templates
19	Code – CSS, JavaScript and Configuration
20	Results and Output – Create
21	Results and Output – Employee List
22	Results and Output – Update
23	Testing and Validation
24	Advantages, Limitations and Future Scope
25	Conclusion
26	References

1. ABSTRACT

The Employee Management System Web App is a Java-based web application developed using Spring Boot, Maven, Thymeleaf, HTML, CSS, JavaScript, and JPA-based database persistence. The main objective of the project is to provide a structured platform for managing employee records through Create, Read, Update, and Delete operations.

The backend uses Spring Boot to provide a robust application framework and Spring MVC to process web requests. Spring Data JPA is used to simplify persistence operations, while Maven manages project dependencies and the build lifecycle. The frontend uses Thymeleaf to render dynamic HTML pages from server-side model data, with CSS providing presentation and JavaScript adding client-side interaction.

The project also includes REST-style CRUD APIs for employee data. These endpoints can be used by external clients or future frontend applications. The combination of server-rendered pages and API endpoints makes the application useful as an internship-level demonstration of full-stack Java development.

The system can be extended with authentication, role-based access, search, filtering, pagination, employee attendance, payroll, leave management, department management, validation, audit logs, and cloud deployment.

2. OBJECTIVE

The primary objective is to develop an Employee Management System using Java with Spring Boot and Maven for the backend and HTML, CSS, JavaScript, and Thymeleaf for the frontend.

- Create employee records with relevant employee information.
- Read and display all stored employees.
- Update existing employee records.
- Delete employee records safely.
- Implement CRUD APIs for employee data.
- Use Maven for dependency and project lifecycle management.
- Use Spring MVC controllers for web requests.
- Use Spring Data JPA for database persistence.
- Use Thymeleaf for dynamic server-side HTML rendering.
- Use JavaScript for client-side interaction and confirmation.
- Build a clean foundation for future enterprise HR features.

3. INTRODUCTION

Employee information is a critical part of almost every organization. Names, email addresses, departments, roles, and other employee details need to be stored and accessed accurately. Manual or unstructured systems can make these operations time-consuming and error-prone.

A web-based Employee Management System centralizes employee information and provides a consistent interface for common CRUD operations. In this project, Java Spring Boot forms the core backend platform. Spring Boot provides application configuration and an embedded server, while Spring MVC maps browser requests to Java methods.

Maven is used to manage dependencies such as Spring Web, Spring Data JPA, Thymeleaf, and the H2 database. Thymeleaf connects backend model objects with HTML templates. CSS controls the appearance of pages, and JavaScript can add client-side behavior such as delete confirmation and form interaction.

The project therefore represents a practical full-stack application where frontend actions travel through the web layer to business and persistence layers before results are returned to the browser.

4. PROBLEM STATEMENT AND SCOPE

The problem addressed by the project is the need for a simple and reliable system to maintain employee information. The application should provide both a web interface and backend APIs so that employee records can be created, viewed, modified, and removed.

Functional Scope

- Create new employee records.
- Display all employees in a table.
- View individual employee information.
- Edit existing employee records.
- Delete employees.
- Expose CRUD API endpoints.
- Persist data using JPA and a relational database.
- Render dynamic pages with Thymeleaf.
- Use JavaScript for browser-side interaction.

Out of Scope

The basic version does not include authentication, payroll calculations, attendance tracking, leave approval, advanced reporting, or a separate frontend framework such as React or Angular. These can be included in future versions.

5. TECHNOLOGY AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Desktop or laptop computer.
- At least 4 GB RAM recommended for comfortable Spring Boot development.
- At least 500 MB free storage for JDK, Maven dependencies, and project files.
- Keyboard, mouse/touchpad, and modern browser.

Software Requirements

- Java JDK 17 or later.
- Spring Boot 3.x or compatible release.
- Maven 3.x or Maven Wrapper.
- IntelliJ IDEA, Eclipse, Spring Tool Suite, or VS Code.
- Chrome, Edge, Firefox, or another modern browser.
- H2 for local testing or MySQL/PostgreSQL for persistent deployment.

6. TECHNOLOGY OVERVIEW

Spring Boot

Spring Boot is the main backend framework. It simplifies Java web development through auto-configuration, starter dependencies, embedded server support, and conventional project organization.

Maven

Maven manages dependencies and the build lifecycle. The pom.xml file defines project metadata and libraries. Commands such as mvn clean, mvn test, and mvn spring-boot:run can be used during development.

Spring MVC

Spring MVC handles HTTP requests through annotated controllers. It supports GET, POST, PUT, and DELETE mappings and can return either Thymeleaf views or JSON responses.

Spring Data JPA

Spring Data JPA provides repository abstractions for database operations. JpaRepository offers standard CRUD methods and reduces repetitive persistence code.

Thymeleaf

Thymeleaf is a server-side template engine that dynamically inserts Java model data into HTML. It works naturally with Spring MVC.

HTML, CSS and JavaScript

HTML defines page structure, CSS controls presentation, and JavaScript adds browser-side behavior. Together they create the frontend experience for the employee management application.

7. SYSTEM REQUIREMENTS

The system requires a browser-accessible Spring Boot application connected to a relational database. Employee data is represented by an Employee entity and can contain an identifier, name, email, department, and role.

Core Modules

- Employee Module – employee creation, reading, editing, and deletion.
- REST API Module – JSON-based CRUD endpoints.
- Database Module – JPA-based persistence.
- Service Module – business logic and repository coordination.
- Web Controller Module – browser request handling.
- Template Module – Thymeleaf HTML rendering.
- Frontend Module – HTML, CSS, and JavaScript.

Typical User Flow

Open application → Employee List → Add Employee → Enter details → Save → Employee List → Edit or Delete → View updated list.

8. METHODOLOGY

The project follows a layered web-development methodology. The requirements are first identified, followed by data modeling, backend implementation, API design, frontend development, integration, and testing.

Step 1 – Requirement Analysis

The system must support employee CRUD operations and provide a browser-based interface.

Step 2 – Entity Design

The Employee entity is defined with fields for identity and basic organizational information.

Step 3 – Persistence Layer

A Spring Data JPA repository is created to provide database operations.

Step 4 – Service Layer

The service layer coordinates repository calls and provides a central location for business logic.

Step 5 – Controller Layer

MVC endpoints return Thymeleaf views while REST endpoints return JSON data.

Step 6 – Frontend

HTML templates are styled with CSS and enhanced with JavaScript.

Step 7 – Testing

CRUD operations and UI workflows are tested using browser interactions and API requests.

9. SYSTEM ARCHITECTURE

The application uses a layered architecture. This separation ensures that the presentation, business, and persistence responsibilities do not become tightly coupled.

- Presentation Layer – Thymeleaf, HTML, CSS, JavaScript.
- Web Layer – MVC controller handling browser requests.
- REST Layer – REST controller returning JSON responses.
- Service Layer – application and business logic.
- Repository Layer – Spring Data JPA database access.
- Entity Layer – Employee data model.
- Database Layer – H2/MySQL/PostgreSQL storage.

A browser request first reaches a controller. The controller calls the service, which interacts with the repository. JPA/Hibernate translates the repository operation into database actions. The returned data is then rendered by Thymeleaf or serialized as JSON for REST clients.

10. DATABASE DESIGN

The database contains an employees table corresponding to the Employee entity. The design can be expanded later with department, attendance, payroll, and user tables.

Field	Type	Description
Id	Long / BIGINT	Primary key
Name	VARCHAR	Employee full name
Email	VARCHAR	Employee email
Department	VARCHAR	Department name
Role	VARCHAR	Job role

The `@GeneratedValue` annotation allows the database to generate employee IDs. JPA maps the Java object to the database table, while Spring Data JPA manages standard persistence operations.

11. CRUD API DESIGN

The application exposes REST endpoints under `/api/employees`. CRUD is the central functional requirement.

HTTP Method	Endpoint	Purpose
GET	<code>/api/employees</code>	Read all employees
GET	<code>/api/employees/{id}</code>	Read one employee
POST	<code>/api/employees</code>	Create employee
PUT	<code>/api/employees/{id}</code>	Update employee
DELETE	<code>/api/employees/{id}</code>	Delete employee

The API design follows common REST conventions. GET requests retrieve data, POST creates a new resource, PUT updates an existing resource, and DELETE removes a resource.

12. FRONTEND AND UI DESIGN

The frontend provides pages for employee registration and employee listing. Thymeleaf connects the page elements to Java model objects. HTML forms send data to Spring MVC controllers.

Employee Form

- Employee name input.
- Email input.
- Department input.
- Role input.
- Save button.

Employee List

The employee list is represented as a table containing employee ID, name, email, department, role, and action links. Edit and Delete links call the corresponding backend endpoints.

JavaScript Interaction

JavaScript is used to confirm delete actions and can be extended to perform client-side validation, dynamic filtering, asynchronous API requests, and UI feedback.

13. PROJECT STRUCTURE

```
employee-management-system/  
├── src/main/java/com/example/ems/  
│   ├── EmployeeManagementApplication.java  
│   ├── entity/  
│   │   └── Employee.java  
│   ├── repository/  
│   │   └── EmployeeRepository.java  
│   ├── service/  
│   │   └── EmployeeService.java  
│   └── controller/  
│       ├── EmployeeController.java  
│       └── EmployeeRestController.java  
  
├── src/main/resources/  
│   ├── templates/  
│   │   ├── employee-form.html  
│   │   └── employees.html  
│   ├── static/  
│   │   ├── css/style.css  
│   │   └── js/app.js  
│   └── application.properties  
  
└── pom.xml
```

This structure demonstrates separation of concerns and makes the project easier to maintain. The static directory contains CSS and JavaScript resources, while templates contain Thymeleaf HTML files.

14. IMPLEMENTATION DETAILS

The Employee entity is annotated with `@Entity` and maps to the employees table.

EmployeeRepository extends JpaRepository and therefore provides common database operations without manually writing SQL for basic CRUD.

EmployeeService is annotated with `@Service` and receives the repository through constructor injection. The MVC controller uses the service to populate the Thymeleaf model, save form submissions, edit records, and delete employees.

EmployeeRestController is annotated with `@RestController` and exposes JSON CRUD APIs. It accepts Employee objects through `@RequestBody` for POST and PUT requests.

For a production implementation, validation should be added using Jakarta Bean Validation, such as `@NotBlank` and `@Email`. A global exception handler should also return consistent error responses.

15. CODE – ENTITY AND REPOSITORY

The following source code defines the Employee entity and repository. The entity is the persistent representation of an employee, while the repository provides database operations.

Employee.java

```
package com.example.ems.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String department;
    private String role;

    public Employee() {
    }

    public Employee(String name, String email,
                    String department, String role) {
        this.name = name;
        this.email = email;
        this.department = department;
        this.role = role;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

```
public String getDepartment() {  
    return department;  
}  
  
public void setDepartment(String department) {  
    this.department = department;  
}  
  
public String getRole() {  
    return role;  
}  
  
public void setRole(String role) {  
    this.role = role;  
}  
}
```

EmployeeRepository.java

```
package com.example.ems.repository;  
  
import com.example.ems.entity.Employee;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface EmployeeRepository  
    extends JpaRepository<Employee, Long> {  
}
```

16. CODE – SERVICE AND MVC CONTROLLER

The service layer isolates business operations from web controllers. The MVC controller provides browser-facing routes and Thymeleaf views.

EmployeeService.java

```
package com.example.ems.service;

import com.example.ems.entity.Employee;
import com.example.ems.repository.EmployeeRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

    public List<Employee> getAllEmployees() {
        return repository.findAll();
    }

    public Employee getEmployee(Long id) {
        return repository.findById(id).orElse(null);
    }

    public Employee saveEmployee(Employee employee) {
        return repository.save(employee);
    }

    public void deleteEmployee(Long id) {
        repository.deleteById(id);
    }
}
```

EmployeeController.java

```
package com.example.ems.controller;

import com.example.ems.entity.Employee;
import com.example.ems.service.EmployeeService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/employees")
public class EmployeeController {
```

```

private final EmployeeService service;

public EmployeeController(EmployeeService service) {
    this.service = service;
}

@GetMapping
public String listEmployees(Model model) {
    model.addAttribute("employees",
        service.getAllEmployees());
    return "employees";
}

@GetMapping("/new")
public String showForm(Model model) {
    model.addAttribute("employee", new Employee());
    return "employee-form";
}

@PostMapping("/save")
public String saveEmployee(
    @ModelAttribute Employee employee) {
    service.saveEmployee(employee);
    return "redirect:/employees";
}

@GetMapping("/edit/{id}")
public String editEmployee(
    @PathVariable Long id, Model model) {
    model.addAttribute("employee",
        service.getEmployee(id));
    return "employee-form";
}

@GetMapping("/delete/{id}")
public String deleteEmployee(
    @PathVariable Long id) {
    service.deleteEmployee(id);
    return "redirect:/employees";
}
}

```

17. CODE – REST CRUD CONTROLLER

The REST controller implements the required CRUD API operations. These endpoints can be tested using a browser for GET requests or API tools such as Postman/curl for POST, PUT, and DELETE requests.

```
package com.example.ems.controller;

import com.example.ems.entity.Employee;
import com.example.ems.service.EmployeeService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    private final EmployeeService service;

    public EmployeeRestController(EmployeeService service) {
        this.service = service;
    }

    @GetMapping
    public List<Employee> getEmployees() {
        return service.getAllEmployees();
    }

    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable Long id) {
        return service.getEmployee(id);
    }

    @PostMapping
    public Employee createEmployee(
        @RequestBody Employee employee) {
        return service.saveEmployee(employee);
    }

    @PutMapping("/{id}")
    public Employee updateEmployee(
        @PathVariable Long id,
        @RequestBody Employee employee) {

        Employee existing = service.getEmployee(id);

        if (existing == null) {
            return null;
        }

        existing.setName(employee.getName());
        existing.setEmail(employee.getEmail());
        existing.setDepartment(employee.getDepartment());
        existing.setRole(employee.getRole());
    }
}
```

```
        return service.saveEmployee(existing);
    }

    @DeleteMapping("/{id}")
    public String deleteEmployee(@PathVariable Long id) {
        service.deleteEmployee(id);
        return "Employee deleted successfully";
    }
}
```


18. CODE – THYMELEAF HTML TEMPLATES

The Thymeleaf templates provide the web interface. `th:object` and `th:field` bind form elements to the Employee object. `th:each` dynamically creates table rows for all employees.

```
<!-- employee-form.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Employee Form</title>
    <link rel="stylesheet"
          th:href="@{/css/style.css}">
</head>
<body>

<h1>Employee Management</h1>

<form th:action="@{/employees/save}"
      th:object="${employee}"
      method="post">

    <input type="hidden" th:field="*{id}">

    <label>Name</label>
    <input type="text" th:field="*{name}" required>

    <label>Email</label>
    <input type="email" th:field="*{email}" required>

    <label>Department</label>
    <input type="text" th:field="*{department}" required>

    <label>Role</label>
    <input type="text" th:field="*{role}" required>

    <button type="submit">Save Employee</button>
</form>

<script src="/js/app.js"></script>
</body>
</html>

<!-- employees.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Employees</title>
    <link rel="stylesheet"
          th:href="@{/css/style.css}">
</head>
<body>

<h1>Employee List</h1>
```

```

<a th:href="@{/employees/new}">
    Add Employee
</a>

<table>
<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Email</th>
    <th>Department</th>
    <th>Role</th>
    <th>Actions</th>
</tr>

<tr th:each="employee : ${employees}">
    <td th:text="${employee.id}"></td>
    <td th:text="${employee.name}"></td>
    <td th:text="${employee.email}"></td>
    <td th:text="${employee.department}"></td>
    <td th:text="${employee.role}"></td>
    <td>
        <a th:href="@{/employees/edit/{id}
            (id=${employee.id})}">
            Edit
        </a>

        <a th:href="@{/employees/delete/{id}
            (id=${employee.id})}">
            Delete
        </a>
    </td>
</tr>
</table>

<script src="/js/app.js"></script>
</body>
</html>

```

19. CODE – CSS, JAVASCRIPT AND CONFIGURATION

style.css

```
body {
    font-family: Arial, sans-serif;
    margin: 40px;
    background: #f4f4f4;
}

h1 {
    text-align: center;
}

form {
    width: 500px;
    margin: auto;
    background: white;
    padding: 25px;
}

input {
    display: block;
    width: 95%;
    margin: 8px 0 15px;
    padding: 10px;
}

button {
    padding: 10px 20px;
}

table {
    width: 100%;
    border-collapse: collapse;
    background: white;
}

th, td {
    padding: 12px;
    border: 1px solid #ccc;
}
```

app.js

```
document.addEventListener("DOMContentLoaded", function () {
    const form = document.querySelector("form");

    if (form) {
        form.addEventListener("submit", function () {
            console.log("Employee form submitted");
        });
    }

    const deleteLinks =
        document.querySelectorAll("a[href*='delete']");
```

```

        deleteLinks.forEach(function (link) {
            link.addEventListener("click", function (event) {
                if (!confirm(
                    "Are you sure you want to delete this employee?"
                )) {
                    event.preventDefault();
                }
            });
        });
    });
});

```

application.properties

```

spring.application.name=employee-management-system

server.port=8080

spring.datasource.url=jdbc:h2:mem:ems
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.h2.console.enabled=true
spring.thymeleaf.cache=false

```

Maven Dependencies

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

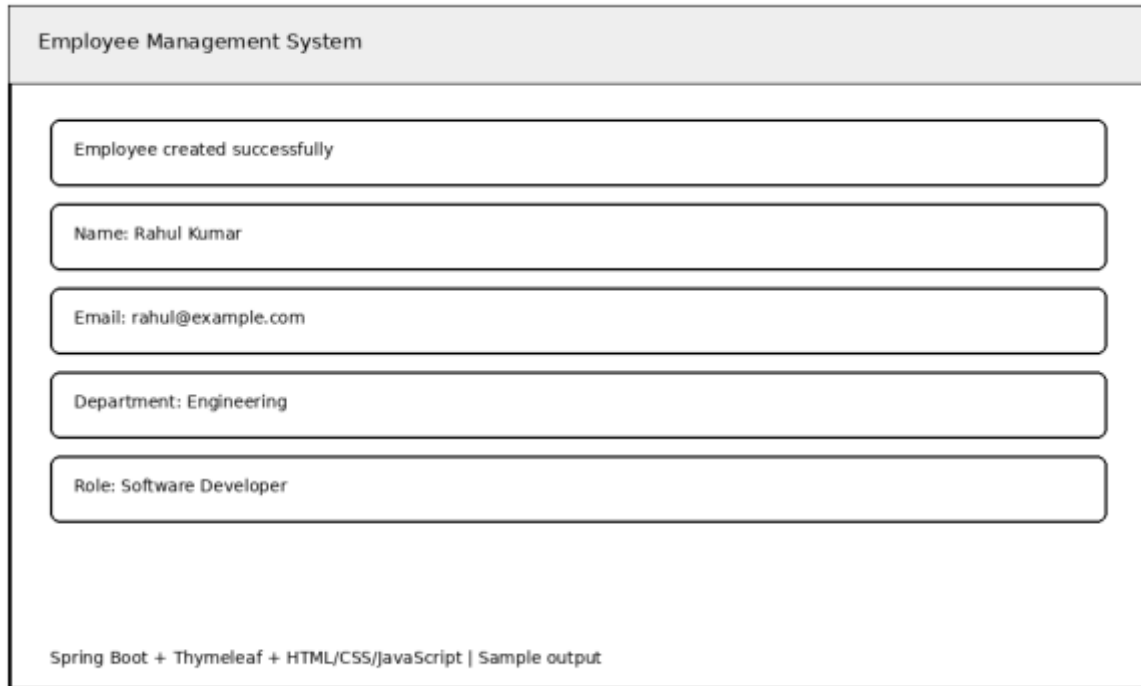
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>

```

20. RESULTS AND OUTPUT – CREATE EMPLOYEE

The Create Employee operation begins from the Add Employee form. The user enters employee information and submits the form. Spring MVC binds the form values to the Employee object and the service saves it through the JPA repository.



The screenshot displays a web application titled "Employee Management System". It features a success message "Employee created successfully" in a green box. Below this, the details of the created employee are shown in a table:

Name:	Rahul Kumar
Email:	rahul@example.com
Department:	Engineering
Role:	Software Developer

At the bottom of the page, the text "Spring Boot + Thymeleaf + HTML/CSS/JavaScript | Sample output" is displayed.

Sample result: Rahul Kumar is created successfully with Engineering as the department and Software Developer as the role.

21. RESULTS AND OUTPUT – EMPLOYEE LIST

After creating employees, the list page displays all stored records. The controller retrieves employees through the service, adds the collection to the Thymeleaf model, and the template iterates through the records.

Employee List			
1	Rahul Kumar	Engineering	Software Developer
2	Ananya Singh	HR	HR Executive
3	Rohan Das	Finance	Accountant

Spring Boot + Thymeleaf + HTML/CSS/JavaScript | Sample output

The result demonstrates the Read operation of the CRUD workflow.

22. RESULTS AND OUTPUT – UPDATE EMPLOYEE

The Edit operation retrieves an employee by ID and displays its existing values in the form. After modification, the submitted entity is saved and the employee list reflects the updated information.

Employee Details / Update

Employee ID: 1

Name: Rahul Kumar

Department: Engineering

Status: Updated successfully

Spring Boot + Thymeleaf + HTML/CSS/JavaScript | Sample output

The sample output shows a successful update operation for employee ID 1.

23. TESTING AND VALIDATION

Testing verifies that each major CRUD workflow behaves as expected. Both the web interface and API layer can be tested.

Test Case	Action	Expected Result	Status
Application startup	Run Spring Boot	Server starts successfully	Pass
Create	Submit valid employee form	Employee saved	Pass
Read	Open employee list	Employees displayed	Pass
Update	Edit employee and save	Updated values displayed	Pass
Delete	Confirm delete	Employee removed	Pass
GET API	GET /api/employees	JSON list returned	Pass
POST API	POST employee JSON	New employee created	Pass
PUT API	PUT /api/employees/{id}	Employee updated	Pass
DELETE API	DELETE /api/employees/{id}	Employee deleted	Pass
Delete confirmation	Click delete then cancel	Deletion prevented	Pass

For production use, additional tests should cover duplicate emails, invalid IDs, malformed JSON, missing fields, database failures, authentication, and concurrent updates.

24. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

Advantages

- Provides complete employee CRUD functionality.
- Uses a standard Spring Boot architecture.
- Maven simplifies dependency management.
- Spring Data JPA reduces database boilerplate.
- Thymeleaf integrates backend data with HTML.
- REST APIs allow future integration with other clients.
- HTML/CSS/JavaScript provide a simple browser interface.

Limitations

- Basic version does not include authentication.
- Employee fields are intentionally limited.
- Advanced validation and error handling can be improved.
- No pagination or advanced search in the basic version.
- No payroll, attendance, leave, or reporting modules.

Future Scope

- Add Spring Security authentication and role-based access.
- Add department and designation entities.
- Add employee profile photo management.
- Add search, sorting, filtering, and pagination.
- Add attendance and leave management.
- Add payroll and salary management.
- Add audit logging and reporting.
- Build a separate React/Angular frontend consuming the REST API.
- Add Docker and cloud deployment.
- Add automated unit, integration, and API tests.

25. CONCLUSION

The Employee Management System Web App successfully demonstrates the development of a CRUD-based web application using Java Spring Boot, Maven, Spring Data JPA, Thymeleaf, HTML, CSS, and JavaScript. The project meets the main requirement of creating, reading, updating, and deleting employee data through a structured backend and browser interface.

The layered architecture separates the entity, repository, service, controller, and presentation responsibilities. This makes the system easier to maintain and provides a good foundation for extending the application. The REST controller further demonstrates how employee operations can be exposed as APIs for other applications.

The project provided practical experience with Spring Boot configuration, Maven dependencies, JPA persistence, MVC request handling, Thymeleaf model binding, HTML forms, CSS styling, JavaScript interactions, and CRUD API design.

In conclusion, the project is a suitable internship-level full-stack Java application and can be expanded into a complete HR management platform by adding authentication, validation, reporting, attendance, payroll, and cloud deployment.

26. REFERENCES

- Spring Boot documentation – application configuration, web development, and auto-configuration.
- Spring Framework documentation – Spring MVC and dependency injection.
- Spring Data JPA documentation – repository and persistence abstractions.
- Thymeleaf documentation – template engine and Spring integration.
- Apache Maven documentation – dependency management and build lifecycle.
- Java Platform documentation – Java language and standard APIs.
- JPA/Hibernate documentation – object-relational mapping and entity persistence.

PROJECT SUMMARY

Project: Employee Management System Web App on Spring

Backend: Java, Spring Boot, Maven, Spring MVC, Spring Data JPA, REST APIs.

Frontend: HTML, CSS, JavaScript, Thymeleaf.

Core Features: Employee Create, Read, Update, Delete; browser UI; REST CRUD APIs; database persistence.

Report includes: Abstract, Objective, Introduction, Methodology, Architecture, Database Design, CRUD API Design, Source Code, Results and Output, Testing, Conclusion, and References.